Jessica Dual-Orbbec Computer Vision

Technical Report: Code Architecture, Datasets, and Matching Models
Rack OPEN/CLOSED (Camera 1) and Furnace Detection (Camera 0)

Laboratory automation vision stack — Jessica workstation
Document date: 2026-10-15 | Implementation roots: `robotics/` and `ros2_ws/src/orbbec_dual_sync/`

1. Purpose and scope

This report documents the computer-vision software used on the Jessica bench to (i) classify the sample rack as **OPEN** or **CLOSED** from Orbbec camera 1, and (ii) localize up to four **furnace** portals on Orbbec camera 0. The system is deliberately **not** a deep neural network for these two tasks: both pipelines use classical template matching (normalized cross-correlation and related cues) supervised by small operator-labeled image banks and multi-process “agents” that capture, label, tune thresholds, train templates, and judge live frames.

Results discussed here (rack OPEN/CLOSED overlays; furnace bounding boxes) come from that classical stack, optionally published through ROS 2 Humble (RoboStack/Pixi) dual camera publishers and a side-by-side dual-view node that paints both overlays in one window.

1.1 Design principles

- **Camera roles are fixed:** cam0 = furnaces; cam1 = rack state.
- **No YOLO required** for rack or furnace (vial YOLO remains optional and off by default).
- **Asymmetric rack logic:** CLOSED is one canonical home pose (strict); OPEN covers many tray poses (looser, multi-exemplar).
- **Fixed furnace slots:** after training, boxes persist even when a robot arm partially occludes a furnace.
- **Agents as processes:** Capture / Label / Algo / Train / Judge (rack) and Capture / Train / Preview (furnace).

2. Hardware, software map, and data flow

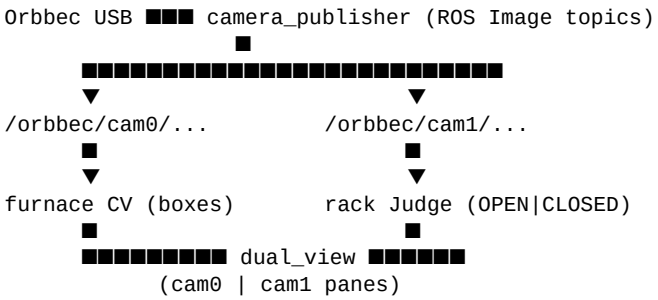
2.1 Sensors

Two Orbbec RGB-D cameras are opened with OpenCV `CAP_OBSENSOR`. Color frames are retrieved with `grab()` + `retrieve(..., CAP_OBSENSOR_BGR_IMAGE)`. Practical color rate is on the order of ~3 Hz per camera when both USB devices stream; publishers are usually rate-limited (~6 Hz timer) and staggered (~20 s) so both devices can enumerate cleanly on Windows.

2.2 Repository layout

Path	Role
robotics/rack_capture_agents.py	MatchScorer + RackStateJudge; capture keepers
robotics/rack_cv_supervisor.py	Supervisor spawning 5 rack agents
robotics/rack_lid_state/	ROI JSON, closed/open templates, keeps/
robotics/furnace_agents.py	Furnace capture / multi-scale NCC / slots
robotics/furnace_state/	furnace.json, templates, captures
ros2_ws/.../orbbec_dual_sync/	Publishers, dual_view, rack/furnace overlays

2.3 End-to-end data flow



Offline training can bypass ROS and open `CAP_OBSENSOR` directly (USB exclusive). ROS publishers and USB openers must not share the same device.

3. Rack OPEN/CLOSED — code architecture

The rack pipeline lives primarily in `rack_capture_agents.py` with orchestration in `rack_cv_supervisor.py`. ROS drawing uses `rack_overlay_draw.classify_whole_rack` and `draw_rack_overlay`, called from `orbbec_rack_overlay` and the cam1 pane of `orbbec_dual_view`.

3.1 Configuration (`rack.json`)

A single JSON file stores the fixed match ROI `(x,y,w,h)=(226,55,137,152)` on a 640×480 cam1 frame, a larger `display_roi` for the on-screen box, optional jar-hole ellipses (legacy focus tool), and pointers to closed/open reference images. The rack mount is assumed static relative to camera 1; lighting drift is handled by refreshing templates, not by re-searching the whole lab.

3.2 Multi-agent supervisor

- **CaptureAgent** — pulls N frames from ROS `/orbbec/cam1/color/image_raw` (BEST_EFFORT) or USB.
- **LabelAgent** — writes `open_*.png` / `closed_*.png` keepers + JSON sidecars; can quarantine mislabeled opens.
- **AlgoAgent** — sweeps gray/edge/hist weights and closed/open thresholds on labeled sets.
- **TrainAgent** — writes `template_roi.png` / `reference_closed.png` or `template_open_roi.png` and `judge_thresholds.json`.
- **JudgeAgent** — `RackStateJudge` smoke accuracy on keepers + live probe.

Operator entry points include `rack_cv_supervisor.cmd bootstrap-closed` and `bootstrap-open` (USB or ROS source).

3.3 Overlay modes

ROS nodes accept `state_mode`: `classify` (default for real CV), `force_open`, or `force_closed`. Force modes are for demos only; production labeling uses classify.

4. Rack “model” — similarity features and decision rule

There is **no neural network** in the rack OPEN/CLOSED path. The “model” is a hand-crafted similarity scorer plus an asymmetric decision tree. This choice favors interpretability, fast CPU inference on Windows, and easy addition of new OPEN tray poses via extra exemplar images.

4.1 MatchScorer features

Given a BGR frame, the scorer crops the match ROI, resizes to a work size (default 160×180), and computes three cues against a reference crop:

- **Gray NCC** — zero-mean normalized cross-correlation on luminance.
- **Edge NCC** — NCC on Canny (or equivalent) edge maps for structure.
- **HSV histogram correlation** — OpenCV `HISTCMP_CORREL` on Hue–Saturation.

A weighted sum forms the match score. Typical trained weights (from AlgoAgent) are approximately gray 0.55, edge 0.30, hist 0.15. A Laplacian variance blur gate rejects soft frames (default blur_min ≈ 35).

```
score = w_gray * NCC(gray) + w_edge * NCC(edges) + w_hist * corr(HSV)
if blur < blur_min:        label = UNKNOWN
elif closed_score >= T_c:  label = CLOSED      # strict home look
elif open_score   >= T_o:  label = OPEN        # best open exemplar
elif closed_score < T_nc and open_score >= T_w:
                           label = OPEN        # not-closed + weak cue
else:                      label = UNKNOWN
```

4.2 Asymmetric JudgeAgent

CLOSED uses a single closed reference (`reference_closed.png` / `template_roi.png`) with a high threshold (stored ~0.75 in `judge_thresholds.json`; code default 0.80). OPEN takes the **maximum** MatchScorer score over a bank of open exemplars (up to ~32 newest `keeps/open_*.png` plus `template_open_roi.png`) with a looser threshold (~0.55). That asymmetry is why one closed home pose stays stable while many tray-extended OPEN looks can be added over time.

5. Rack dataset

5.1 Contents and naming

Labeled images live under `robotics/rack_lid_state/keeps/`. Filenames encode the ground-truth class: `closed_*.png` and `open_*.png`. JSON sidecars store source path, agent name, and timestamp. Rough inventory at report time:

Subset	Approx. count	Notes
open_*.png keepers	~79	Multiple tray-out poses + live bursts
closed_*.png keepers	~48	Home / retracted rack
reference_closed.png	1	Canonical CLOSED ROI template
template_open_roi.png	1	Primary OPEN ROI template
judge_thresholds.json	1	T_c, T_o, weights, max exemplars

5.2 How data is collected

- Live USB or ROS bursts while the operator holds a physical state (tray in / tray out).
- Operator screenshots of the overlay (crops are auto-embedded into a blank 640×480 frame so ROI scoring still works).
- Quarantine of open keepers that score too highly against the closed template (mislabels).

5.3 OPEN pose diversity

OPEN is not a single geometry. Exemplars include mid-tray extension, lower-tray extension with white-lid jars, earlier tray-out poses, and live sequences at ~0.3–0.5 s spacing. Each new pose is added by Capture+Label+Train without replacing the closed template. Leave-one-out exclusion is used when judging a keeper that is also an open exemplar, to avoid trivial self-matches.

5.4 Evaluation protocol

JudgeAgent smoke mode scores all `closed_*` / `open_*` keepers and reports accuracy, UNKNOWN rate, and confusion. Live probes print `closed_score` and `open_score` per frame. Example observed values after recent OPEN training: live tray-out frames with open≈0.94 (OPEN); closed home probes with closed≈0.77 (CLOSED).

6. Furnace detection — code architecture (Camera 0)

Furnaces appear only on camera 0 as nested rectangles: dark outer frame, light inner border, dark cavity. Implementation: `furnace_agents.py` with persistent state in `furnace_state/`. ROS dual-view paints cam0 via `furnace_overlay_draw.paint_furnace_overlay` (parameter `enable_furnace_cv:=true`).

6.1 Agents

- **CaptureAgent** — interval grabs to `furnace_state/captures/cam0_*.png` (ROS or USB).
- **TrainAgent** — scans recent readable captures, multi-scale match, writes slots + per-slot templates.
- **PreviewAgent** — draws boxes and labels `furnace0`… (optional live window).

6.2 Slot file (`furnace.json`)

After the latest retrain for a changed cam0 viewpoint, two slots were locked (up to four allowed):

Slot	xywh (x,y,w,h)	Score	Label
0	(221, 132, 63, 50)	0.822	furnace
1	(287, 134, 63, 50)	0.705	furnace

Frame size 640×480; match threshold currently 0.52; scales {0.75, 0.9, 1.0, 1.15, 1.3}. Flag `keep_occluded: true` means trained boxes remain drawn when the robot covers a furnace — the product requirement was to still box and label occluded portals rather than drop them.

6.3 Viewpoint change procedure

When cam0 is physically moved, old captures are archived, a fresh burst is recorded, and TrainAgent rewrites `furnace.json`. This avoids stale xywh from the previous mount geometry.

7. Furnace “model” and dataset

7.1 Multi-scale template matching

Again there is **no DNN**. A seed crop (`seed_crop.png` / `template_furnace.png`) showing one furnace portal is matched across the full cam0 frame at several scales using normalized correlation (OpenCV template matching / NCC family). Candidates are filtered by score threshold, aspect-ratio tolerance (±~35% vs seed), and non-maximum suppression (IoU≈0.25). At most four peaks become slots; per-slot ROI crops are saved under `furnace_state/templates/` for later refresh.

```
for scale in scales:
    tmpl_s = resize(template, scale)
    response = matchTemplate(frame_gray, tmpl_s, TM_CCOEFF_NORMED)
    peaks = local_maxima(response) where score >= threshold
slots = NMS(peaks, iou=0.25)[:4]
persist xywh + score to furnace.json
```

7.2 Dataset

Asset	Approx. count	Role
Current captures/	~16–17	Post-retrain viewpoint burst
Archived cam0_*.png	~193	Prior viewpoint (kept for audit)
seed / template_furnace.png	1	Appearance prior
templates/furnace{i}.png	2+	Per-slot crops
preview_latest.png	1	Annotated sanity check

Corrupt PNGs (truncated writes) are skipped during train scans (libpng read errors). Train prefers the capture that maximizes detection count and score sum among the newest readable files.

7.3 Why not YOLO here?

Furnaces are nearly fixed in the camera frustum, have a strong rectangular appearance prior, and must remain labeled under occlusion. Template slots meet those constraints with minimal labeling cost. A detector could be swapped in later; the overlay API only needs xywh + label lists.

8. ROS dual view, overlays, and observed results

8.1 Dual publishers and UI

Package `orbbec_dual_sync` launches cam0/cam1 publishers, then (on timers) `orbbec_dual_view` and `orbbec_rack_overlay`. Dual view composes cam0|cam1: left pane furnace boxes, right pane rack OPEN/CLOSED. Preview uses tkinter+Pillow on Windows because conda OpenCV HighGUI is unreliable inside `ros2 launch` children. Image transport uses raw `sensor_msgs/Image` with BEST_EFFORT QoS (no cv_bridge dependency in the hot path).

8.2 Representative quantitative results

Task	Observation	Interpretation
Rack CLOSED probe	closed≈0.77, open≈0.59 → CLOSED	Strict home template wins
Rack OPEN live burst	closed≈0.55, open≈0.94 → OPEN	Multi-exemplar open bank
New OPEN screenshot	closed≈0.17, open≈1.00 → OPEN	Fullframe embed + exemplars
Furnaces (new view)	2 slots @ 0.82 / 0.70	Multi-scale NCC lock-in

8.3 Failure modes addressed in engineering

- **force_open pinning** hid real mistakes — default returned to classify.
- **OPEN pose missing** — add keepers via bootstrap-open / supervisor.
- **Cam0 moved** — archive captures and retrain furnace slots.
- **USB vs ROS exclusive lock** — never open CAP_OBSENSOR while publishers own the device.
- **Dual-view "waiting cam0/cam1"** — subscriptions need live topics; tk must stay on the main thread.

9. Operational runbook and mathematical detail

9.1 Common commands

```
cd C:\Users\Ceder-ALAB\Desktop\Jessica\Jessica\robotics
rack_cv_supervisor.cmd bootstrap-closed --source ros --n 8
rack_cv_supervisor.cmd bootstrap-open   --source usb --camera 1 --n 8
anaconda3\python.exe furnace_agents.py train --max-scan 40 --disk-only
anaconda3\python.exe furnace_agents.py stop

cd ..\ros2_ws
launch_ros_dual_sync.cmd
  (enable_furnace_cv + rack_state_mode:=classify)
```

9.2 NCC definition (MatchScorer)

For zero-mean vectors **a**, **b** of equal length (flattened work-size patches):

NCC(a,b) = (a′ · b′) / (■a′■ ■b′■), where a′ = a – mean(a).

Scores near 1.0 indicate strong structural agreement with the reference. Histogram correlation is bounded roughly in [–1, 1] and is down-weighted relative to gray/edge so illumination shifts do not dominate geometry.

9.3 Quality controls

- Warm-up grabs (≥60) before trusting Orbbec frames.
- Blur gate (Laplacian variance) before OPEN/CLOSED decisions.
- Reject-cap when capturing closed keepers to avoid flooding disk.
- Leave-one-out when an open keeper is also an exemplar.
- Corrupt PNG skip in furnace train scans.

10. Limitations, future work, and conclusion

10.1 Limitations

- Classical matching is sensitive to large lighting changes and camera refocus; refresh templates when the scene drifts.
- OPEN coverage is only as good as the exemplar bank — novel tray poses may return UNKNOWN until captured.
- Furnace detector currently locked two of four possible portals after the latest viewpoint change; more may appear when visible/unoccluded.
- Not a learned detector: generalization outside the Jessica mount geometry is not claimed.
- Windows USB/ROS exclusivity and DDS large-image quirks require careful process hygiene.

10.2 Plausible upgrades

- Lightweight CNN/YOLO for furnaces if mounts become less rigid.
- Depth-channel cues for tray extension (OPEN) vs home (CLOSED).
- Active learning loop: UNKNOWN live frames auto-queued for operator label.
- Calibration tool to click four furnace portals once after a camera move.

10.3 Conclusion

The Jessica dual-Orbbec vision results for rack OPEN/CLOSED and furnace boxes are produced by a transparent, agent-driven classical CV stack: ROI template similarity with gray/edge/histogram fusion and an asymmetric judge for the rack; multi-scale NCC with fixed slots for furnaces. Datasets are small, operator-curated PNG banks under `rack_lid_state/keeps` and `furnace_state/`. ROS dual-view integrates both overlays for operators without requiring deep learning runtimes on the critical path. Extending performance is primarily a data problem—add OPEN exemplars or retrain furnace slots after viewpoint changes—rather than a model-architecture problem.

Document control. Generated from the live Jessica robotics tree. Primary implementers: multi-agent rack/furnace tooling in `robotics/`; ROS packaging in `orbbec_dual_sync`. This PDF is intended for lab handoff, design review, and onboarding—not as a substitute for reading the source modules cited above.